



**INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO**

**INGENIERÍA EN SISTEMAS COMPUTACIONALES
BASE DE DATOS**

Práctica 1 “Modelo Entidad Relación”

Hurtado Avilés Gabriel

Equipo:

1. Altamirano Rodríguez Valeria.
2. García Fernández David.
3. Roldán Rumbo Angela Itzel.

16 de Septiembre 2026

Ejercicio 1: Control de versiones con Git y GitHub.

Parte A. Investigación.

1. ¿Qué es un sistema control de versiones y qué problema concreto resuelve en un trabajo en equipo?

Los sistemas de control de versiones son software que ayudan a realizar un seguimiento de los cambios realizados en el código a lo largo del tiempo. Sin este, los desarrolladores tienen la tentación de mantener varias copias de código en su equipo. Esto es peligroso porque es fácil cambiar o eliminar un archivo en la copia incorrecta del código, lo que podría perder el trabajo. Los sistemas de control de versiones resuelven este problema administrando todas las versiones del código, pero presentando al equipo una sola versión a la vez.

Asimismo, mejora la eficacia en los trabajos colaborativos, las versiones se sincronizan y se asegura que los cambios realizados de diversos usuarios no choquen entre sí, se mantiene un historial por lo que pueden hacer diversos cambios sin el miedo que pueda provocar algo irreversible, esto también ayuda a que podamos hacer retroalimentación o apoyo a otros compañeros.

2. ¿Cuál es la diferencia entre Git y GitHub?

Git es un sistema de control de versiones que realiza un seguimiento de los cambios en los archivos. Le permite trabajar en su propia copia de un proyecto, registrar un historial de los cambios y combinar su trabajo con los cambios de otras personas de forma segura.

GitHub se basa en Git mediante el hospedaje de los proyectos de Git, denominados repositorios, en la nube, así como la adición de herramientas de planeación y colaboración que facilitan a los equipos trabajar juntos.

3. Definir las siguientes palabras:

- **Repositorio:** Es un tipo de almacenamiento digital, se utiliza para realizar y administrar cambios en el código fuente (central). En el mismo se guardan los cambios que se realizaron, quién los hizo, en qué momento y las versiones del código.
- **Confirmación (commit):** Es un punto de guardado del código, al momento de agregar los cambios en git se le agrega un breve texto con los cambios realizados para tener un idea posteriormente, al igual que se sabe quien realizó el cambio.
- **rama (branch):** Es una línea de desarrollo independiente dentro de un repositorio. Permite trabajar en nuevas funcionalidades o corregir errores sin afectar al código principal.
Se utiliza mucho en el desarrollo colaborativo, ya que facilita la organización y la integración de cambios de manera estructurada.
Es como cuando modificas recetas, primero lo intentas si funciona y queda bien, ya cambias la receta original.
- **fusión (merge):** Posterior de la revisión de la rama se agregan los cambios al código principal, básicamente se fusionan la rama principal con la nueva.

Si en la rama de un compañero se cambió el fondo a morado y fue aceptado, entonces se agrega a la rama principal, por lo que el fondo ahora es morado.

- **Conflicto de fusión:** Los conflictos de fusión surgen cuando múltiples desarrolladores que trabajan en el mismo proyecto realizan cambios conflictivos en la misma parte del código dentro de sus respectivas ramas. Al intentar fusionar las ramas, el sistema detecta que se han realizado cambios contradictorios en la misma ubicación.

Que en la misma parte de código la rama de uno cambie el fondo a rojo, y justo en la rama de otro integrante lo cambia el fondo a azul.

- **Pull Request:** Es una petición que el propietario de un fork de un repositorio hace al propietario del repositorio original para que este último incorpore los commits.

Un fork es la copia de un proyecto que no es tuyo, pero en el cual haces cambios.

- **Archivo .gitignore:** Es un archivo de texto colocado en el directorio raíz del repositorio Git. Contiene patrones que indican a Git qué archivos o directorios debe ignorar. Estos patrones pueden personalizarse para adaptarse a las necesidades de los proyectos, ayudando a mantenerlo bien organizado. Justo se puede ver cómo las cosas que subimos en redes sociales, nosotros elegimos que van a ver de nosotros los demás y que otras no.

- **Archivo readme:** Es un documento que explica qué es el proyecto, cómo usarlo, cómo instalarlo y cualquier otra información relevante para los usuarios o colaboradores. Es como un manual o carta de presentación.

4. ¿En qué consiste un flujo de trabajo basado en ramas y por qué el código se revisa entre pares antes de fusionarse?

Son secuencias de pasos organizados en la que un equipo desarrolla o modifica el código. En cuanto a la revisión previa, básicamente es para detectar problemas que la persona que programó no vea, así evitando problemas en el código principal.

Parte B. Práctica.

```
100219229@ValeAlt MINGW64 ~/OneDrive - Instituto Politecnico Nacional/Escritorio/Prueba/reposito-Practica1.8 (main)
$ git log --oneline --graph --all
* de0f7a7 (HEAD -> main, origin/main, origin/HEAD) Merge pull request #1 from ValeAR1/mejora-readme
* 355c17a (origin/mejora-readme, mejora-readme) Agrega seccion de herramientas utilizadas en el README y se pone a prueba la rama
* 10d4148 Descripción de lo que lleva el documento de la practica
* ec99e05 Agrega archivo de investigación-de-bases al repositorio
* 521ec28 Agrega sección de objetivo de la practica en el README
* b957fdc Agrega archivo .gitignore para ignorar archivos del sistema y del editor
* 8c4ea7e Agrega información e índice de la practica al README
* aa51f17 Initial commit
```



Ejercicio 2. El sistema gestor en un contenedor: Docker.

Parte A. Investigación.

1. ¿Qué es un contenedor y en qué se diferencia de una máquina virtual, en términos de arranque, tamaño y aislamiento?

Un contenedor es un entorno aislado donde se ejecuta una aplicación con todo lo que necesita para funcionar: librerías, dependencias, configuración básica y procesos necesarios. La aplicación se empaqueta de forma que pueda ejecutarse de manera parecida en distintos entornos.

En las máquinas virtuales funcionan mediante virtualización, por lo que se ocupa hipervisores que son una capa de software para el servidor físico, es la que divide la RAM, ROM, potencia, etc. (Recursos que vienen de la máquina física), a cada una de las VM, sin interferir unas con otras. Las VM pueden tener diferente sistema operativo dependiendo de lo que ocupe. Por lo que las máquinas virtuales tienen más tamaño, se tarda más en arrancar ya que pasa desde la base (hardware físico), al sistema anfitrión y posteriormente al hipervisor y luego a la VM. Por ello también, el aislamiento es mejor, ya que al tener un sistema operativo interno, es una máquina independiente.

En cuanto a los contenedores, estos comparten el mismo sistema operativo que el anfitrión, por lo cual es más rápido, el tamaño es menor también. Sin embargo el aislamiento es menor, ya que dependen del mismo núcleo o kernel.

2. Defina:

- **Imagen:** Plantilla que tiene todo lo necesario para ejecutar el sistema.

- **Contenedor:** lugar donde se encuentra.
 - **Volumen:** Donde se almacenan los datos del contenedor (data).
 - **Puerto publicado:** Dirección de la conexión.
3. ¿Por qué el volumen es indispensable y qué ocurre exactamente si no lo declara?
 Porque es donde se almacenan los datos, si no se declara se perdería información y no se pueden recuperar.

Parte B. Práctica.

The screenshot shows a VS Code editor with a file named `docker-compose.yml` open. The file contains the following configuration:

```

services:
  postgres:
    image: postgres:17
    container_name: postgres173CV231AG02026
    environment:
      POSTGRES_DB: practical1
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5433:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    restart: unless-stopped
  
```

Below the editor, the `TERMINAL` tab shows the execution of the following commands:

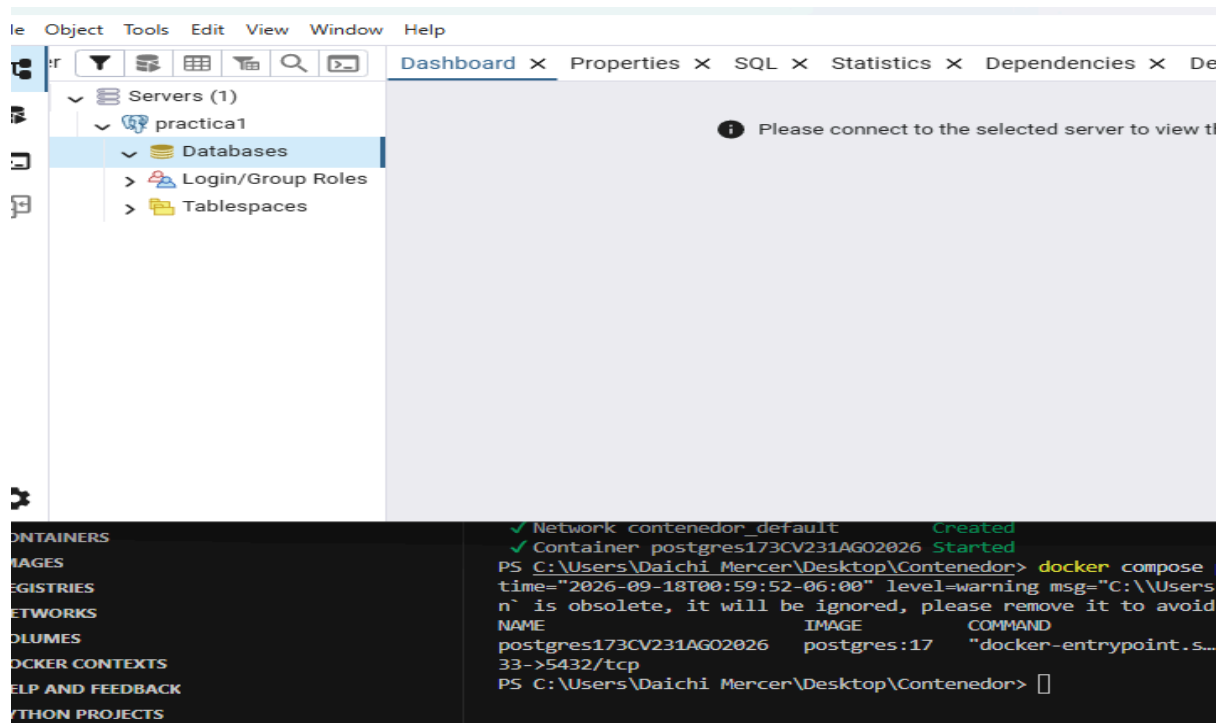
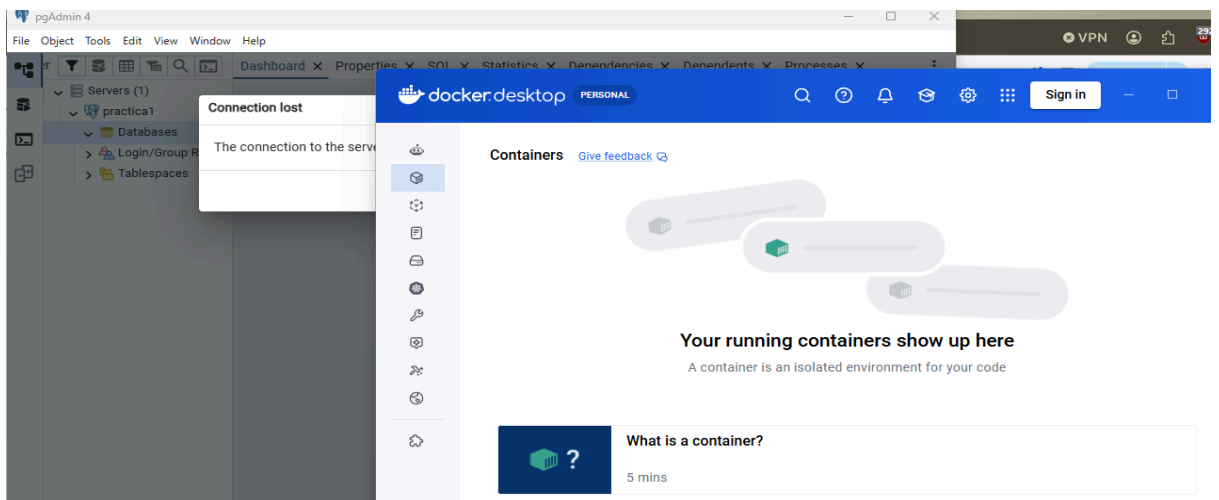
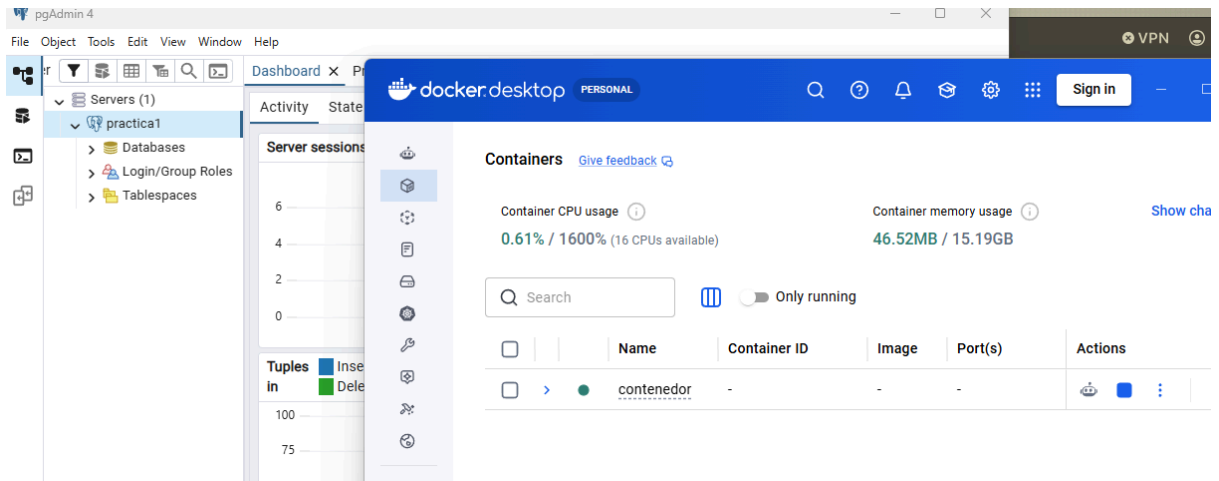
```

PS C:\Users\Daichi Mercer\Desktop\Contenedor> docker compose up -d
time="2026-09-18T00:42:31-06:00" level=warning msg="C:\\Users\\Daichi Mercer\\Desktop\\Contenedor\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 2/2
✓ Network contenedor_default Created
✓ Container postgres173CV231AG02026 Started
PS C:\Users\Daichi Mercer\Desktop\Contenedor> docker compose ps
time="2026-09-18T00:42:49-06:00" level=warning msg="C:\\Users\\Daichi Mercer\\Desktop\\Contenedor\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME                IMAGE              COMMAND                  SERVICE  CREATED      STATUS      PORTS
postgres173CV231AG02026 postgres:17        "docker-entrypoint.s..." postgres  15 seconds ago Up 14 seconds 0.0.0.0:5433->5432/tcp
  
```

The screenshot shows a 'Register - Server' dialog box with the following fields and values:

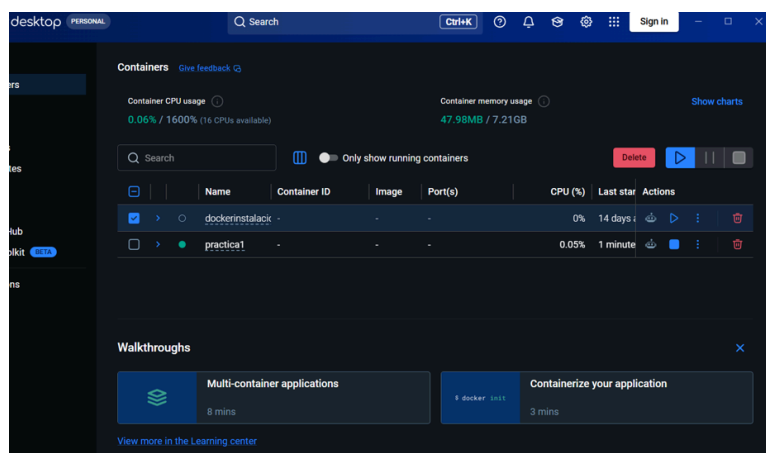
- Host name/address:** localhost
- Port:** 5433
- Maintenance database:** postgres
- Username:** postgres
- Kerberos authentication?** ☒
- Password:** (masked with dots)
- Save password?** ☐
- Role:** (empty)

At the bottom of the dialog, there are buttons for 'Close', 'Reset', and 'Save'. A 'Community Support' link is also visible in the bottom right corner.



```
C:\Users\anger> cd /OneDrive/Desktop/practica1 & docker-compose.yml
1 version: '3.8'
2
3 services:
4   postgres:
5     image: postgres:17
6     container_name: practica1
7     environment:
8       POSTGRES_DB: postgres
9       POSTGRES_USER: postgres
10      POSTGRES_PASSWORD: 442005
11     ports:
12       - "5432:5432"
13     volumes:
14       - ./data:/var/lib/postgresql/data
15     restart: unless-stopped

Problems Output Debug Console Terminal Ports
PS C:\Users\anger\OneDrive\Desktop\lol> docker --version
Docker version 29.7.2, build a7dcaa6
PS C:\Users\anger\OneDrive\Desktop\lol> cd..
PS C:\Users\anger\OneDrive\Desktop> cd practica1
PS C:\Users\anger\OneDrive\Desktop\practica1> docker compose up -d
time="2026-09-18T23:07:30-06:00" level=warning msg="C:\\Users\\anger\\OneDrive\\Desktop\\practica1\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine; check if the path is correct and if the daemon is running: open //.\\pipe\\dockerDesktopLinuxEngine: The system cannot find the file specified.
PS C:\Users\anger\OneDrive\Desktop\practica1> docker compose up -d
time="2026-09-18T23:08:52-06:00" level=warning msg="C:\\Users\\anger\\OneDrive\\Desktop\\practica1\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 2/2
  ✓ Network practica1_default Created 0.1s
  ✓ Container practica1 Started 0.8s
PS C:\Users\anger\OneDrive\Desktop\practica1>
```



```
Learn more at https://docs.docker.com/go/debug-cl1/
PS C:\Users\anger\OneDrive\Desktop\practica1> docker compose down
time="2026-09-18T23:16:10-06:00" level=warning msg="C:\\Users\\anger\\OneDrive\\Desktop\\practica1\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] down 2/2
  ✓ Container practica1 Removed 0.5s
  ✓ Network practica1_default Removed 0.3s
PS C:\Users\anger\OneDrive\Desktop\practica1>
```

List of databases									
Name	Owner	Encoding	Locale Provider	Collate	Ctype	Locale	ICU Rules	Access privileges	

postgres	postgres	UTF8	libc	en_US.utf8	en_US.utf8				
practica1	postgres	UTF8	libc	en_US.utf8	en_US.utf8				
template0	postgres	UTF8	libc	en_US.utf8	en_US.utf8				=c/postgres
+									
gres									postgres=CTc/post
template1	postgres	UTF8	libc	en_US.utf8	en_US.utf8				=c/postgres
+									
gres									postgres=CTc/post
(4 rows)									
(END)									

What's next:

Try Docker Debug for seamless, persistent debugging tools in any container or image → `docker debug practica1`
 Learn more at <https://docs.docker.com/go/debug-cli/>

```
PS C:\Users\anger\OneDrive\Desktop\practica1> docker compose down
time="2026-09-18T23:16:10-06:00" level=warning msg="C:\\Users\\anger\\OneDrive\\Desktop\\practica1\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] down 2/2
✓ Container practica1      Removed      0.5s
✓ Network practica1_default Removed      0.3s
PS C:\Users\anger\OneDrive\Desktop\practica1> docker compose up -d
time="2026-09-18T23:16:45-06:00" level=warning msg="C:\\Users\\anger\\OneDrive\\Desktop\\practica1\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 2/2
✓ Network practica1_default Created      0.1s
✓ Container practica1      Started      0.4s
PS C:\Users\anger\OneDrive\Desktop\practica1>
```

The screenshot shows the Docker Desktop interface with the PostgreSQL database 'Practica1' selected. The left sidebar shows the 'Volumes' section with a volume named 'entorno_pgdata' created 25 minutes ago, with a size of 38.5 MB. The right sidebar shows the 'SQL' tab with the following SQL script:

```
-- Database: Practica1
-- DROP DATABASE IF EXISTS "Practica1";
CREATE DATABASE "Practica1"
WITH
  OWNER = postgres
  ENCODING = 'UTF8'
  LC_COLLATE = 'en_US.utf8'
  LC_CTYPE = 'en_US.utf8'
  LOCALE_PROVIDER = 'libc'
  TABLESPACE = pg_default
  CONNECTION LIMIT = -1
  IS_TEMPLATE = False;
```

The bottom section shows the 'SQL' tab with a message: "-- No SQL could be generated for the selected object."

